EchoPin v0.4: update speech

About 13½ minutes (1147 words at a relaxed pace). The same words are in each slide's speaker notes, where ▶ means advance.

How to read this. [CLICK] means advance the slide (each click starts the next animation). The time after each heading is where you should be when that slide ends. If you're running long, skip slide 12 (*See it*); it saves about 45 seconds.

For code questions, use `CODE_REFERENCE.md`: every function mentioned here, with its file and line, and short answers to the questions you're most likely to get.

---

1. Title (ends 0:45)

Good morning, everyone. The last time I stood here, I showed you EchoPin: an app that keeps a group together without GPS, without servers and without accounts. Today I want to tell you how EchoPin grew up, and why it is now safer, not less safe, even though it can talk to a server. Three things changed: real accounts, sealed emergency records, and a central console. And then we spent a lot of time trying to break all of it.

---

2. Where we started (ends 1:45)

First, what did not change. **[CLICK]** EchoPin still never knows where you are. There is no latitude or longitude anywhere in the code. Distance comes from Bluetooth signal strength, measured on your own phone. Phones still talk directly to each other, so it works on a mountain with no signal. And every message is still encrypted and signed. **[CLICK]** None of that changed. Everything I'm about to show you sits on top of it.

---

3. The brief (ends 2:30)

So why change anything? Because we were asked for four things. **[CLICK]** Real user accounts, with more security than identifying phones by their IMEI number. A history of sessions that the right people can check in an emergency. Reports and diagnostics from every phone in one place. And backups. **[CLICK]** In the words of the brief: more security than the system's IMEI, and reports across all systems.

4. The big idea (ends 3:30)

Here's how we did it. **[CLICK]** Phones still talk to each other exactly as before. When there's signal, an outbox on the phone forwards things to a central server: accounts, sealed session records, reports and diagnostics. Responders and admins see that in a console. The outbox is the clever bit: if you're offline, nothing is lost. It waits, and sends when it can. **[CLICK]** And the server is optional. If an organisation doesn't set one up, the app runs exactly like before: local, offline, no account.

---

5. Identity (ends 4:30)

Now, the IMEI question. Why not just identify phones by their IMEI? **[CLICK]** Because other apps can read it. **[CLICK]** Anyone who reads it can copy and replay it. **[CLICK]** Modern Android and iPhones mostly don't let apps see it at all. **[CLICK]** And you can't revoke it. Instead, every install makes its own device key. The private half never leaves the phone. **[CLICK]** When you sign in, the phone signs a short statement with that key, and the server checks it within five minutes. So if someone steals your refresh token, it's useless without your actual phone.

---

6. Two locks (ends 5:45)

The most sensitive thing we store is the emergency record of a session. **[CLICK]** The record says who was there, when someone pressed SOS, who checked in. Never coordinates. **[CLICK]** It's sealed inside a lock that only responders can open, using a custodian key that lives on their devices, never on the server. **[CLICK]** And that's sealed again inside a lock only the server can open. **[CLICK]** So the server on its own can't read a record. Someone who steals the database and the server's keys still can't read one. A responder in a real emergency can: the server opens its layer, and the console opens the rest.

---

7. Break-glass (ends 6:45)

Opening a record is what we call break-glass, like the fire alarm. **[CLICK]** You must give a reason. **[CLICK]** The server writes that reason into the audit log before it decrypts anything, so there's no way to read a record without leaving a trace. **[CLICK]** Every admin gets an email straight away. **[CLICK]** Then the responder's own device

opens the inner lock. **[CLICK]** And staff accounts must turn on two-step sign-in before the console will even answer them.

---

8. We attacked our own code (ends 7:45)

Then we tried to break it. **[CLICK]** The first security review found ten faults and fixed every one. **[CLICK]** A second pass closed the eleven risks we had first accepted. **[CLICK]** And then testing the real app against a live server found five more bugs that no automated test had caught. **[CLICK]** The important part: none of them let anyone read a sealed record.

---

9. Highlights (ends 9:00)

A few highlights, in plain words. **[CLICK]** The server used to trust an IP address the client could simply make up, which let attackers dodge rate limits and fake the audit log. Fixed. Someone could register your email just after you did and have you activate their password. Fixed: confirming now needs your password too. Anyone could lock you out by typing wrong passwords. Now your own phones skip that lock. **[CLICK]** The server no longer reveals who has an account, not even through timing. Passwords that appear in public data breaches are refused, and only five characters of a hash leave the server to check. And the audit log is now signed with a secret key, so even someone with full database access can't rewrite history.

---

10. Tested for real (ends 9:45)

How do we know it works? **[CLICK]** Thirty out of thirty checks against a real running server, plus over two hundred and eighty automated tests across the app, the server and the shared protocol. **[CLICK]** And we didn't stop at unit tests: a real compiled server, a real database, and the real app, clicked through on a hidden screen.

---

11. Found by real testing (ends 10:45)

That last step paid off. Five bugs. **[CLICK]** The two-step setup dialog crashed on every platform. **[CLICK]** The console crashed right after a responder opened a record. **[CLICK]** One SOS was being counted three times, once per phone that saw it. **[CLICK]** With exactly one real member and one outsider, the membership check could brand the real member as the fake. **[CLICK]** And our own test runs were writing into the app's real data folder. Every one is fixed, and every one now has a test.

12. See it (ends 11:30)

Here's the real app. **[CLICK]** An admin without two-step is turned away. **[CLICK]** A session where someone uploaded a record without being in it: flagged, "not proven a member". **[CLICK]** And the custodian key, imported on a responder's device and never uploaded. **[CLICK]** These are straight from the test run.

---

13. Running it (ends 12:15)

Running it costs nothing. **[CLICK]** GitHub holds the code and keeps the server awake. Render runs the server. Supabase holds the database and the encrypted backups. Brevo sends the sign-in codes. **[CLICK]** The keys never go in the code; the server refuses to start without them. And the data sits in Frankfurt today, but the same image runs in a South African region if POPIA needs it.

---

14. Limits and next steps (ends 13:00)

I want to be honest about what's left. **[CLICK]** Before real use, we need to create the custodian key, turn on two-step for every account around the server, test Bluetooth between real phones, and decide where the data lives. **[CLICK]** Next, we'd like verified-account badges inside sessions, a real deployment, and a field trial with a real group.

---

15. Thank you (ends 13:30)

EchoPin still doesn't know where you are. Now it can also help the right people find out what happened, and only them, and only with a reason everyone can see. Thank you. I'm happy to take questions, including about any line of code behind this.